



Chapter 1

Manipulating Raw Data

IN THIS CHAPTER

» Obtaining data

» Defining the forms of data

» Making data access reliable

.....

Data scientists not only work with data but also spend considerable time pursuing data from various sources. Sometimes this pursuit resembles that of a detective ferreting out clues from arcane sources. Consequently, any in-depth conversation about data, as you see it in later chapters of this minibook, must begin with the simple idea of obtaining data in a manner that will prove useful for analysis later. The acquisition of raw data in various forms is the focus of this chapter.

If you find it surprising that a data scientist doesn't automatically know where to find a particular piece of information, consider the vastness of data today. Looking for a needle in a haystack is easy compared to locating that much-needed piece of data from all the sources that a data scientist has available. In some cases, you find that you must generate data with specific characteristics to perform tests that validate assumptions about raw data, so the data you need may not even exist until you create it. The first section of this chapter looks at raw data sources.

Recognizing the forms of data is also important because you rarely find data in the form you need. For example, you can find a great deal of raw textual data in various places and lightly formatted data in others. After a while, you recognize the patterns of data and the processes used to obtain it in a specific form. The second section of this chapter views data formats from a raw data perspective, which may not represent the final data format used for an analysis.

Because you rarely perform an analysis once, the data you obtain must be reliable in that you can be certain that the data will appear from a particular source, in an expected form, and with the characteristics that you need. The final section of this chapter describes reliability as it applies to raw data.

Defining the Data Sources

To perform an analysis, you must have data. However, data must have a source, and the source you rely on affects all sorts of factors that also affect your analysis. Even though you can categorize data sources in a wide variety of ways, the following sections look at data as coming from the following:

- **Locally:** On a hard drive attached to your system or your network. The main advantages of this data source are speed and reliability.
- **Web or other online sources:** The data is located somewhere other than a system that you control directly in most cases. The main advantages of this data source are diversity and freshness (how current the data is).
- **Dynamically generated:** The application creates the data in some manner. The main advantages of this data source are consistency and completeness (meaning that you won't find any missing data unless you specifically add it).
- **Synthetically generated:** You create the data you use according to criteria defined by a software script. (An example is the `make_classification` function from Scikit-learn: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_classification.html.) Synthetically generating data can help you test your algorithms or prove a

theory. The main advantage is that you have full control of the data and its characteristics.

Obtaining data locally

In many cases, the data you need to work with won't appear within a library, as the toy datasets do, for example, in the Scikit-learn library. Real-world data usually appears in a file of some type. A flat file presents the easiest kind of file to work with. The data appears as a simple list of entries that you can read one at a time, if desired, into memory. Depending on the requirements for your project, you can read all or part of the file.

A problem with using native Python techniques is that the input isn't intelligent. For example, when a file contains a header, Python simply reads it as yet more data to process, rather than as a header. You can't easily select a particular column of data. The pandas library used in the sections that follow makes it much easier to read and understand flat-file data. Classes and methods in the pandas library interpret (parse) the flat-file data to make it easier to manipulate.



REMEMBER The least formatted and therefore easiest-to-read flat-file format is the text file. However, a text file also treats all data as strings, so you often have to convert numeric data into other forms. A comma-separated value (CSV) file provides more formatting and more information, but it requires a little more effort to read. At the high end of flat-file formatting are custom data formats, such as an Excel file, which contains extensive formatting and could include multiple datasets in a single file.

The following sections describe these three levels of flat-file dataset. ([Chapter 4](#) of this minibook contains examples of how to access them.) These sections assume that the file structures the data in some way.

For example, the CSV file uses commas to separate data fields. A text file might rely on tabs to separate data fields. An Excel file uses a complex method to separate data fields and to provide a wealth of information about each field. You can work with unstructured data as well, but working with structured data is much easier because you know where each field begins and ends.

Working with flat files

A flat file is simply a file that contains data in some form, normally as text. The overriding characteristic of a flat file is that it contains a single data entry, normally a table. You commonly see flat files with these characteristics:

- Each data row is separated by a carriage return, line feed, or combination of the two.
- Each column is separated by a tab or other control character that isn't used for rows.
- The data isn't formatted in any way, so strings aren't normally quoted.
- The file may or may not contain a header row to identify the columns.
- The file normally relies on pure text, such as ASCII or UTF-8 formatted characters.

A flat file represents the simplest available method of transferring data between any two entities, even when they're different platforms or if the devices would normally prove incompatible. The problems for the data scientist using flat files are numerous, especially when the flat file comes within documentation:

- The flat file may not rely on control characters to define rows and columns; it may use some sort of positional format instead.
- Interpreting some data proves impossible, such as whether 1 represents a numeric value or a string.
- Missing data is nearly impossible to locate and add in.
- Parsing the file can be difficult or impossible when the original file contains mistakes.

You use flat files when simplicity and ease of data transfer override other considerations. The ability to generally view the data in a form that humans can recognize and understand directly is also a big plus. However, you also need to consider the additional time required to process this type of file.

Using organized databases

Databases come in many forms. You also get different interpretations of the term depending on the experiences of the person describing a database. For some people, a CSV file is an example of a database, rather than a flat file, because of the organization and formatting that a CSV file provides. However, other people consider a CSV a kind of flat file because it doesn't go far enough in formatting the data and in providing some sort of standardized access method. At the other end of the spectrum are relational databases that include their own programming language, diagramming, and extensive control over data format. The point is that databases are organized methods of storing data that have these characteristics:

- Rows and columns are distinctly identified using a specific methodology.
- Some form of data formatting is employed so that it becomes possible to separate the string form of 1 from the integer form of 1.
- Some form of column identification is provided so that it becomes possible to perform tasks like comparing files of the same type.
- The file may contain metadata to characterize the file content and parsing requirements.
- Because the files are organized, finding and fixing many data issues, such as missing data, become easier.



TIP

The preceding isn't a complete list of the characteristics found in all organized data sources, but it's a good start. You might find all sorts of additional features that include security and other management needs. However, a general rule of thumb is that as the number of

database features increase, so does complexity and the need for specific parsing mechanisms. You can parse a CSV using a general text processor if necessary, but you can't say the same for an Excel file or a file used by a SQL database. In fact, in some cases, you need a specific parser for each version of a database product.

Complexity isn't the only potential issue when using organized databases. You can also encounter the following issues, which make using an organized database significantly more difficult:

- The file sizes are usually larger than a corresponding flat file, which means using more resources to manage them.
- Some databases only work on a specific operating system platform, which means you can't use them on all the devices in your organization.
- The appearance of multiple tables and other objects within a single file complicates parsing.
- The data isn't understandable by a human in its raw form.
- Creating bridges between various files can prove difficult, necessitating the use of transformations and other coding tricks.

Relational and NoSQL databases

The vast majority of data used by organizations rely on relational databases because these databases provide the means for organizing massive amounts of complex data in a manner that makes the data easy to manipulate. The goal of a database manager is to make data easy to manipulate; the focus of most data storage is to make data easy to retrieve.

Relational databases accomplish both the manipulation and data retrieval objectives with relative ease. However, because data storage needs come in all shapes and sizes for a wide range of computing platforms, many different relational database products exist. In fact, for the data scientist, the proliferation of different Database Management Systems (DBMSs) using various data layouts is one of the main prob-

lems you encounter with creating a comprehensive dataset for analysis.

The one common denominator among many relational databases is that they all rely on a form of the same language to perform data manipulation, which does make the data scientist's job easier. The Structured Query Language (SQL) lets you perform all sorts of management tasks in a relational database, retrieve data as needed, and even shape it in a particular way so that the need to perform additional shaping is unnecessary.

In addition to standard relational databases that rely on SQL, you find a wealth of databases of all sorts that don't have to rely on SQL. These Not Only Structured Query Language (NoSQL) databases are used in large data storage scenarios in which the relational model can become overly complex or can break down in other ways. The databases generally don't use the relational model. Of course, you find fewer of these DBMSes used in the corporate environment because they require special handling and training. Still, some common DBMSes are used because they provide special functionality or meet unique requirements. The process is essentially the same for using NoSQL databases as it is for relational databases:

1. Import required database engine functionality.
2. Create a database engine.
3. Make any required queries using the database engine and the functionality supported by the DBMS.

The details vary quite a bit, and you need to know which library to use with your particular database product. For example, when working with MongoDB (<https://www.mongodb.org/>), you must obtain a copy of the PyMongo library (<https://api.mongodb.org/python/current/>) and use the MongoClient class to create the required engine.

Consuming freeform databases

Freeform databases can contain multiple tables, each of which has a different format. In addition, the data within a table need not necessarily following a specific format. Because you can't gauge the format by using a header, these databases require a great deal more formatting. Products such as askSam

(<https://asksam.software.informer.com/>) commonly see use for freeform informational databases. Accessing askSam would require a special parser. (You can likely use the same technique applied to relational databases as described at

<https://www.dummies.com/programming/big-data/data-science/data-science-how-to-use-python-to-manage-data-from-relational-databases/>.)

Unlike other forms of data storage, a freeform database may not even use the table convention for storing information. You may find that it uses a hierarchical format instead, which means relying on special coding to move from record to record. The simple need to know what data the file contains and in the order in which it appears can prove difficult to meet. However, freeform storage can also prove to be incredibly space efficient, and you can use it to customize the data store so that the database becomes more flexible than just about any other means of storing data.



REMEMBER Most people would categorize eXtensible Markup Language (XML) and JavaScript Object Notation (JSON) as types of freeform databases. Both use hierarchical storage techniques and provide extreme flexibility. As long as you don't violate the few rules that each of these formats requires you to observe, the systems generally work as you might think they should. However, the flexibility these file formats provides can become a problem because the files can literally contain anything. To combat this issue, XML files can rely on an XML Schema Definition (XSD) file

(<https://www.tutorialspoint.com/xsd/index.htm>) and JSON can

rely on a JSON Schema file

(https://www.tutorialspoint.com/json/json_schema.htm).

Another important consideration is that some freeform databases rely on a different disk storage format than their in-memory presentation; the hierarchy or other in-memory form is built from data as it appears on disk. The use of this approach means that you can create a robust in-memory presentation that requires less disk storage space than conventional databases require. Because freeform databases have significantly fewer rules than other data storage techniques, presenting a solid list of characteristics, pros, and cons is impossible.

Using online data sources

The amount of data available online defies conception. In fact, you can't even visualize it because it boggles the imagination. The fact that each day sees more data added to online sources than many people could consume in a lifetime says much about online data. At some point, you use online data or you find yourself hopelessly outmatched by others who do. With this reality in mind, the following sections discuss online sources of raw data — some of which needs considerable manipulation before it provides any sort of useful information.

Accessing publicly available datasets

Governments, universities, nonprofit organizations, and other entities often maintain publicly available databases that you can use alone or combined with other databases to create big data for machine learning. For example, you can combine several Geographic Information Systems (GIS) to help create the big data required to make decisions such as where to put new stores or factories. The machine learning algorithm can take all sorts of information into account — everything from the amount of taxes you have to pay to the elevation of the land your store sits on (which can contribute to making your store easier to see).

FINDING YET MORE DATA ON DATA SCIENCE CENTRAL

A data scientist must have resources for locating data because no one person can possibly know about every source. Many of the resources you find online cover mainstream topics that you might find helpful in enabling your data service, but that might fall short of affording an ultimate resource. Data Science Central

(<https://www.datasciencecentral.com/>) provides access to a relatively large number of data science experts who tell you about the most obscure facts of data science. One of the more interesting blog posts appears at

<https://www.datasciencecentral.com/profiles/blogs/huge-trello-list-of-great-data-science-resources>.

Data Science Central points you to a Trello list (<https://trello.com/>) of some truly amazing resources. Navigating the huge list can be a bit difficult, but the process is aided by the treelike structure that Trello provides for organizing information. You want to meander through this sort of list when you have time and simply want to see what is available. The categories include the following (with possibly more by the time you read this book):

- Data news
- Data business people track
- Data journalist track
- Data padawan track
- Data scientist track
- Statistics
- R
- Python
- Big data and other tools
- Data
- Others

The best part about using public data is that it's usually free, even for commercial use (or you pay a nominal fee for it). In addition, many of

the organizations that created them maintain these sources in nearly perfect condition because the organization has a mandate, uses the data to attract income, or uses the data internally. When obtaining public source data, you need to consider a number of issues to ensure that you actually get something useful. Here are some of the criteria you should think about when making a decision:

- The cost, if any, of using the data source
- The formatting of the data source
- Access to the data source (which means having the proper infrastructure in place, such as an Internet connection when using Twitter data)
- Permission to use the data source (some data sources are copyrighted)
- Potential issues in cleaning the data to make it useful for machine learning

Scraping data from websites

It's important to understand that many of the data sources you use come from online content in the form of web pages and other web sources. *Scraping data* is the process of extracting useful data from a web page, while removing the nondata elements, such as tags. One of the better products for performing this task is BeautifulSoup (<https://www.crummy.com/software/BeautifulSoup/>). The example in the “[Scraping Textual Datasets from the Web](#)” section of Book 4, [Chapter 4](#) tells you how to use this library in a practical way.

Relying on data from APIs

An Application Programming Interface (API) relies on a system of requests and responses to serve data. A client makes a request and a server provides a response. The specifics of each API vary, and you find that the strategies can become quite complex. The underlying technology for various APIs also differs. However, from a data perspective, you can expect to see the information sent and retrieved in a standards-oriented manner using technologies such as

- XML

- JSON
- Binary (generally only for private APIs)



Pure text messaging is uncommon and perhaps even nonexistent today. The XML formats can become quite specialized. For example, when using the Simple Object Access Protocol (SOAP) to interact with an API, you use a specially formatted XML document that follows the SOAP messaging format (see <https://www.w3schools.com/xml/xml:soap.asp> for details). When working with an API, you must fully understand the techniques for interacting with it, in addition to later transforming the data to meet your needs.

Binary formats such as the Common Object Request Broker Architecture (CORBA) may seem outdated, but you see them used for private APIs for a number of reasons, including security and performance. You can often transmit binary data at significantly higher speeds than text data of the same content. The article at <https://www.guru99.com/comparison-between-web-services.html> discusses the whole alphabet soup of technologies used for web services, including:

- Representation State Transfer (REST)
- SOAP
- CORBA
- Distributed Common Object Model (DCOM)
- Java Remote Method Invocation (RMI)

Of the binary formats, CORBA seems to be the most popular given that Microsoft fully embraces SOAP for its web offerings today. You can get a better overview of CORBA at <https://www.sciencedirect.com/topics/computer-science/common-object-request-broker-architecture>. The article

at

<http://www.conference.org/proceedings/www2002/alternate/395/index.html>

provides a more detailed view of why CORBA might be a good choice when working with certain kinds of APIs.

No matter which kind of API you use and the type of data it serves, you generally need to do the following:

1. Transform the data from its transmitted form to a form suitable for processing.
2. Remove any extraneous information used as part of the transmission process.
3. Clean the data to remove undesirable elements.
4. Validate that the data is complete and hasn't suffered transmission errors.
5. Translate the data into a form that matches the other data used for your analysis.

Gaining access to private data

You can obtain data from private organizations such as Amazon and Google, both of which maintain immense databases that contain all sorts of useful information. In this case, you should expect to pay for access to the data, especially when used in a commercial setting. You may not be allowed to download the data to your personal servers, so that restriction may affect how you use the data in a machine learning environment. For example, some algorithms work slower with data that they must access in small pieces.

The biggest advantage of using data from a private source is that you can expect better consistency. The data is likely cleaner than from a public source. In addition, you usually have access to a larger database with a greater variety of data types. Of course, it all depends on where you get the data.

Employing dynamic data sources

Dynamic data sources are those that change over time. For example, the weather doesn't remain static — it may rain today and not tomorrow. The probability of rain changes, which affects how you plan outside activities. The current weather predictions are always dynamic because they're always changing. However, once the weather occurs and becomes historical in nature, it also becomes a static data source. The weather, once past, doesn't change. If there was a tornado on a certain day, the tornado doesn't somehow go away in the future — there is always a tornado for that day.



REMEMBER As with the weather, many data sources start as dynamic data sources and become static data sources in the future. Consequently, when viewing data for use within an application, you must always consider whether the data is dynamic or static. Static data is easier to handle because it never changes. Dynamic data requires that you perform updates on a specific schedule and then perform your analysis again if you are to get any benefit from the analysis. With these differences between dynamic and static in mind, the following sections discuss various forms of raw dynamic data and consider how you might handle them as part of an analysis.

Monitoring the user

Users receive a large share of the monitoring associated with dynamic data. Because this monitoring is usually surreptitious to avoid biasing the data, it's more akin to spying. People spy on each other for all sorts of reasons — everything from performing marketing studies to conducting efficiency analysis. Much of this spying is benign; some of it is even helpful to the user. For example, sleep studies spy on the sleeper to determine whether modern technology can assist in reducing harmful sleep habits. The reason for monitoring (spying on) the user varies, but the result is normally data that reflects habits of some sort that prove helpful in predicting future actions. Even recommender sys-

tems, those aids that tell you that one item goes with another item or that people who purchased a particular item also bought another, rely on the study of buying habits.



REMEMBER When it comes to users, you need to consider issues beyond simple monitoring and analysis. The article “AI is finding out when the person using your account isn’t you” (see <https://thenextweb.com/problem-solvers/2018/07/13/authentication-cybersecurity/>) points out a particular problem with current thinking. It discusses the use of behavioral analytics as a means for discovering the fraudulent use of an ID, but behavioral analytics don’t consider that human behaviors can change suddenly because of catastrophic events, such as the loss of a loved one. Fortunately, the article also discusses other approaches, such as the use of facial recognition and biometrics. However, no matter how you perform monitoring (or spying, as the case might be), the data received is apt to contain flaws that you must ferret out as part of the analysis.

For the most part, humans do change slowly (see “Change Doesn’t Happen Overnight: It Happens in These Five Stages” at <https://www.forbes.com/sites/amymorin/2014/03/17/change-doesnt-happen-overnight-it-happens-in-these-five-stages/> for details), so behavioral analytics work much of the time. However, you want to maintain the outlook that human behavior is quite dynamic and you need to constantly look for those changes that signal a major life event if your job is to predict the future.

Obtaining generated data

Your existing data may not work well for some data analysis scenarios, but that doesn’t keep you from creating a new data source using the old data as a starting point. For example, you might find that you have

a customer database that contains all the customer orders, but the data isn't useful for your particular analysis because it lacks tags required to group the data into specific types. One of the new job types that you can expect to create is people who massage data to make it better suited for a particular analysis type, including the addition of specific information types such as tags.



REMEMBER Data analysis of all sorts has a significant effect on your business. The article at

<https://www.computerworld.com/article/3007053/how-machine-learning-will-affect-your-business.html> describes some of the ways in which you can expect machine learning to change how you do business. One of the points in this article is that machine learning typically works on 80 percent of the data. In 20 percent of the cases, you still need humans to take over the job of deciding just how to react to the data and then act upon it. The point is that using machine learning to manipulate your data saves money by taking over repetitious tasks that humans don't really want to do in the first place (making them inefficient). However, machine learning doesn't get rid of the need for humans completely, and it creates the need for new types of jobs that are a bit more interesting than the ones that machine learning has taken over. Also important to consider is that you need more humans at the outset until the modifications they make train the algorithm to understand what sorts of changes to make to the data.

Whether you work with AI, machine learning, deep learning, or perform some sort of other data analysis, as a data scientist, you may also need to generate test data. Some packages and libraries include data generators for this purpose. You can also find data generators online that perform *mocking*, which is the simulation of a data source using fake data that reflects the data you expect from the actual source. The Mockaroo (<https://mockaroo.com/>) and Generate Data

(<https://www.generatedata.com/>) sites are examples of this sort of data generation.

Considering other kinds of data sources

Your organization has data hidden in all kinds of places. Recognizing the data as data can be a problem, though. For example, you may have sensors on an assembly line that track how products move through the assembly process and ensure that the assembly line remains efficient. Those same sensors can potentially feed information into an algorithm because they could provide inputs on how product movement affects customer satisfaction or the price you pay for postage. The idea is to discover how to create mashups that present existing data as a new kind of data that lets you do more to make your organization work well.



REMEMBER Big data can come from any source, even your email. A recent article discusses how Google uses your email to create a list of potential responses for new emails. (See the article at <https://www.semrush.com/blog/deep-learning-an-upcoming-gmail-feature-that-will-answer-your-emails-for-you/>.) Instead of having to respond to every email individually, you can simply select a canned response at the bottom of the page. This sort of automation isn't possible without the original email data source. Looking for big data in specific locations will blind you to the big data sitting in common places that most people don't think about as data sources. Tomorrow's applications will rely on these alternative data sources, but to create these applications, you must begin seeing the data hidden in plain view today.

Some of these applications already exist, and you're completely unaware of them. The article at <https://www.microsoft.com/en-us/research/video/the-master-algorithm-how-the-quest-for-the->

[ultimate-learning-machine-will-remake-our-world/](https://www.youtube.com/watch?v=8Ppqep-KAYI&feature=youtu.be) makes the presence of these kinds of applications more apparent. (You can watch just the video at <https://www.youtube.com/watch?v=8Ppqep-KAYI&feature=youtu.be>.) By the time you complete the video, you begin to understand that many uses of machine learning are already in place and users already take them for granted (or have no idea that the application is even present).

Considering the Data Forms

Previous sections of the chapter have discussed the forms data appears in from an overview perspective. The form of data you receive affects the following:

- How you interact with it
- The level of information you can expect to derive from it
- Issues related to data complexity
- Time required to process and manicure it
- Biases that could appear within it

The following sections provide a detailed view of the various data forms that you can expect to encounter. They break these forms into three main groups: pure text, formatted text, and binary. You might see data in other forms, but not often and usually not in a meaningful form.

Working with pure text

Pure text consists of the alphanumeric characters in the character set you use, such as American Standard Code for Information Interchange (ASCII) or Unicode Transformation Format 8-bit (UTF-8), and specific control characters, such as tab, linefeed, and carriage return. The reason for this extreme limit is to make the data created with pure text universally acceptable by the greatest number of devices and operating systems in existence.

With compatibility in mind, standard ASCII

(<http://www.asciitable.com/>) is perhaps the most universal character set of all. However, even with these limits, ASCII isn't universal because some very old systems use Extended Binary Coded Decimal Interchange Code (EBCDIC) (see <https://pediaa.com/difference-between-ascii-and-ebcdic/> for details). When you compare an ASCII table to an EBCDIC table (<http://www.astrodigital.org/digital/ebcdic.html>), you see that the two encodings are incompatible.



TIP

About now, you may be wondering why this whole encoding issue is important given that most modern computers can use extended ASCII (a 256-character version of original ASCII) and UTF-8 without any problem at all. The problem is that the data you need might not be from a modern machine, especially if your analysis has a historical basis to it. Consequently, you need to know that pure text, even with its extreme limitations, is hardly the universal transfer media that you might think it would be. When working with data, even pure text, you must be ready to deal with the unexpected.

Pure text doesn't necessarily come in a specific format, either. You can order data in a file using a number of approaches. Therefore, you need to know how the data is organized before you can process it. Here are a few of the most common approaches to data organization:

- **Freeform:** This text appears in a semiformatted state using control characters to separate fields and another set of control characters to separate rows.
- **Text-based freeform:** This form is similar to freeform but relies on special text combinations instead of control characters to separate fields and rows. For example, a ZZ pair could signal the end of a field, while a ZZZ triplet could signal the end of a row. In most cases, you find this form used only with specialized applications or in-house uses.
- **Positional:** This text doesn't rely on any control characters to separate fields, but instead relies on the size of each field to determine the beginning

and ending of a field. Rows are separated using control characters, normally the carriage return, line feed, or a combination of the two.

- **Continuous:** This text that doesn't use control characters for any purpose, but simply relies on field size to determine every aspect of data format.



TECHNICAL
STUFF

Lest you think that this list is complete, it's not. Point-of-Sale (POS) terminals are notorious for using truly unconventional data formats, for example. The article at

<https://www.acceleratedanalytics.com/blog/2010/01/15/top-questions-about-point-of-sale-data-analysis/> offers some clues as to just how convoluted the supposedly pure text data provided by POS terminals can become. As the article reveals, you can't simply import the data into your Windows system and view it in Excel.

Of course, the biggest problem with pure text is that you get just the data — no context, no description, and especially no metadata. To use pure text formats, you must know about the source used to create the data, which means intimate knowledge of the originator as well. In some cases, pure text simply can't provide what you need to perform a complete data analysis.

Accessing formatted text

Formatted text can take on a number of forms. You begin with pure text, but then add clues as to the formatting of the data. Here are some things that you find in a formatted text file that you won't find in a pure text file:

- Contains headers to describe the fields
- Contains metadata to tell you about the data source and other data features
- Uses quoting to make strings and numbers different
- Treats numbers with decimal points as floating point, even when the number lacks a decimal portion or the decimal portion is 0
- Uses True and False (or some variant) to define Boolean values

- Uses keywords to denote data categories
- Specifies field boundaries by using delimiters
- Specifies rows by using carriage return, linefeed, or both

Not every formatted text file contains all these features, and some formatted text files rely on other characteristics to amplify the information you need. The point is that the underlying data is supported by additional, nondata information that tells you about the data so that you can interpret it with greater precision.

When you begin working with highly formatted text files, such as XML, JSON, and HTML, you start to see patterns and hierarchies. For example, the tags and other organizational aids used with these kinds of file aren't part of the data; instead, they're part of the metadata. You use them to see the construction and texture of the data. Automated processing designed to interpret these organizational aids can create datasets of extreme complexity that allow you to perform advanced analysis with a higher degree of confidence.

The use of stylesheets and other data input aids also increases the consistency of highly formatted text files by imposing rules for validating new data. Ensuring the absolute integrity of any data resource is impossible, but the use of validation tools does reduce the incidence of incorrect data and make the data more reliable.



REMEMBER The positive aspects of formatting come at a price, unfortunately. As the data format becomes more complex and the tools for working with it become more useful, the ability to transfer the data anywhere you want diminishes. In addition, the processing requirements for such data increases, increasing the likelihood that you need a more capable device to even see the data correctly, much less process it. There is no free lunch. The increased use of formatting con-

veys more information but also requires more resources to handle and reduces flexibility.

Deciphering binary data

Binary data comes in many forms and it doesn't just pertain to older technologies such as CORBA. Graphics are binary, as is music and many other forms of nontextual information.

Nontextual data generally comes only as binary data, but you find exceptions. For example, Scalable Vector Graphics (SVG) come as XML files (https://www.w3schools.com/graphics/svg_intro.asp) that describe what to draw rather than the drawing itself. Theoretically, you can use the same techniques with SVG that you use with any XML file to perform an analysis of the graphic image it describes, rather than rely on deciphering binary data. All graphics files that fall into this category are vector graphics (based on math) rather than raster graphics (based on individual pixels) (see https://vector-conversions.com/vectorizing/raster_vs_vector.html for details).

Things get more complicated when you want to analyze the rendering of a vector graphic because now you have a raster graphic rendering to deal with. For example, you might want to know why a vector graphic produces a moiré pattern (<http://mathworld.wolfram.com/MoirePattern.html>) at one resolution and not another. The point is that you may find that you started with text, but now are working with binary data despite your desire to avoid doing so by using a textual data format.



REMEMBER When working with text, binary formats often became popular for a number of reasons:

- Transmitting the data is more efficient than pure text.

- The data can contain formatting information inline, so the formatting doesn't get lost.
- Securing the data from prying eyes is quite easy.
- The use of checksums and other binary strategies can increase reliability and make the data self-repairing.
- You can include information that isn't possible with text formats, such as placing graphics and text together (as in a PDF).

Binary data became unpopular for a number of reasons that include complexity, difficulty of processing, and platform specificity. However, you see binary data of this sort today and you'll likely continue to see it in the future. In some cases, you really do need to use a binary format.

When working with binary data, you need to consider all sorts of features that you may not find in other file types, such as a signature identifying the kind of binary data. The file may contain structural information and processing hints. You may find data in several formats residing in the same file. In short, binary data simply requires more processing than normal data because it doesn't appear in a form that humans understand. Consequently, when working with binary data, you must know something about the application that generated the data and have specifications available that describe the data format.

Understanding the Need for Data Reliability

Data, like everything else, has a certain reliability. The problem is determining what reliability means when it concerns data. In most cases, to ensure that you have reliable data, you must consider these issues:

- The data source remains available.
- Static data doesn't change.
- Dynamic data is updated as often as needed to ensure that it doesn't get stale.
- Errant data is corrected, but with a change in dataset version number so that you know it has changed.

- The data files aren't corrupted in some manner (and not just from a virus or adware, but also from natural and unnatural sources).
- Alternative sites provide data access when a host site becomes unavailable.
- Someone is actually maintaining the data (it isn't orphaned in some way).
- The creator of the data source is fully identified.
- A third party has vetted the data to ensure its integrity.



REMEMBER When your data meets all these criteria, you have data that is reasonably reliable. To summarize, the data must remain accessible in a form that you expect and without any outside tampering to be useful. Otherwise, you can't be sure that any analysis you perform using the data has meaning. It's hard to hit the bull's-eye when the target constantly changes position.

Of course, these criteria talk about only the actual data file and its raw content, to an extent. The data itself must meet certain characteristics to be reliable. What you want in this case is data that has been

- Examined thoroughly during a peer review
- Validated to meet appropriate standards
- Collected with good scientific principles and statistical means in mind
- Relies on best practices for meeting a particular need
- Reflects reality with regard to specific conditions

In most cases, simply knowing that you have data is not enough. You need to know that the data targets something specifically oriented toward your analysis needs. Collecting emails from various people is useful only when those people are part of a target group for your analysis. Otherwise, you begin drawing incorrect conclusions from the data, and your analysis is no longer valid. One of the most important aspects of reliable data, then, is peer review, which can help ensure that bias and other issues don't cloud the judgment of those collecting the data. The "[Considering the Five Mistruths in Data](#)" section of Book

6, [Chapter 2](#) discusses the sorts of issues that can make reasonable-looking data unacceptable.

Table of contents collapsed